Solution: Buy Two Chocolates

Problem statement

You are in a chocolate shop where each chocolate has a price, as listed in an array `prices`. With a certain amount of money in your pocket, you aim to purchase `exactly two` chocolates.

You would like to `minimize` the combined cost of these chocolates so that you are left with as much money as possible, without going into debt.

Calculate the amount of money you'll have left after making the purchase. If it's not possible to buy two chocolates without `overspending`, return the `original amount` of money you had.

Examples

Example 1:

- **Input**: prices = [2, 5, 3, 9], money = 10
- **Expected Output**: 5
- **Justification**: The optimal choice is to buy chocolates priced at 2 and 3, costing a total of 5. Therefore, you're left with 10 - 5 = 5.

Example 2:

- **Input**: prices = [10, 20, 15, 5, 25, 30, 35, 40], money = 50
- **Expected Output**: 35
- **Justification**: The cheapest two chocolates has cost 5 and 10. Purchasing these costs 15 in total. Thus, you are left with 50 - 15 = 35.

Example 3:

- **Input**: prices = [7, 1, 5, 3], money = 3
- **Expected Output**: 3
- **Justification**: It's impossible to buy two chocolates without going into debt, so you keep your original amount of 3.

Solution

To solve this problem, we first sort the array of chocolate prices in ascending order. This arrangement allows us to easily identify the two cheapest chocolates, as they will be positioned at the beginning of the sorted array. By purchasing these two chocolates, we ensure that we spend the minimum possible amount of our budget, thereby maximizing the leftover money. Subtracting the total cost of these two chocolates from the initial amount of money provides us with the desired result. This approach is both straightforward and efficient, as it leverages sorting to quickly solve the problem.

Step-by-step Algorithm

1. **Sort** the array of chocolate prices in ascending order.
2. **Select** the first two elements of the sorted array as they represent the cheapest chocolates.
3. **Calculate** the sum of the prices of these two chocolates.
4. **Subtract** this sum from the initial amount of money to find the leftover money.
5. **Return** the leftover money. If the sum of the prices of the two cheapest chocolates exceeds the initial amount of money, it indicates that it's not possible to buy two chocolates without going into debt; hence, return the original amount of money in such a case.

Algorithm Walkthrough

Given the input prices = [10, 20, 15, 5, 25, 30, 35, 40], money = 50:

1. **Sort the prices array**: First, we sort the array of chocolate prices to easily identify the cheapest options. The sorted array becomes [5, 10, 15, 20, 25, 30, 35, 40].

2. **Identify the two cheapest chocolates**: The two cheapest chocolates are the first two elements of the sorted array, which are priced at 5 and 10.

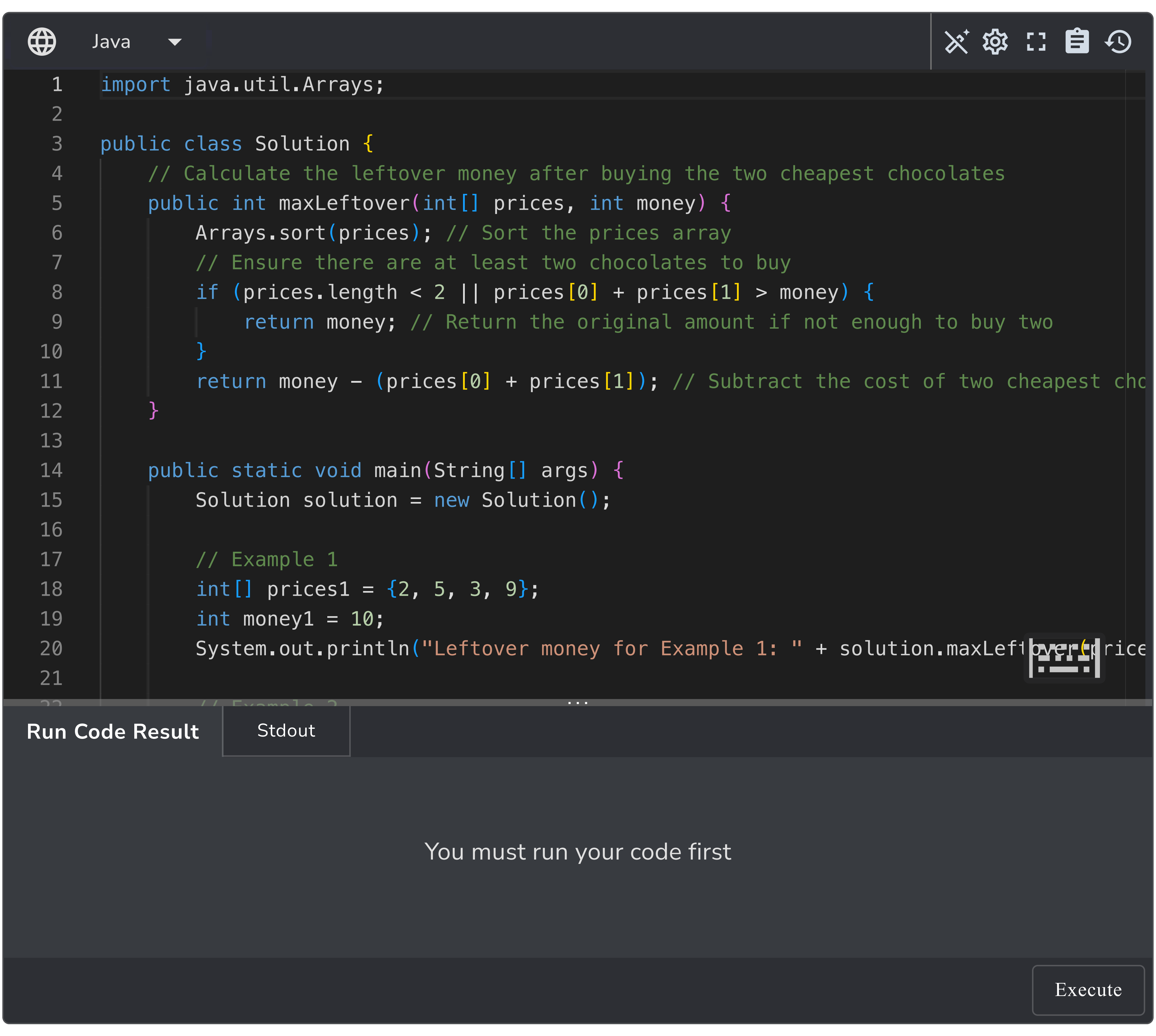
3. **Calculate the total cost of these chocolates**: We add the prices of these two chocolates together, resulting in a total cost of 5 + 10 = 15.

4. **Determine if the purchase is possible within the budget**: Before making the purchase, we need to ensure that the total cost of the two chocolates does not exceed our budget of 50. In this case, 15 is less than 50, which means the purchase is within our budget.

5. **Calculate the leftover money**: Since the total cost of 15 is within our budget, we subtract this amount from our initial money to determine how much will be left after the purchase. Therefore, we perform the calculation 50 - 15 = 35.

6. **Return the result**: The algorithm returns 35, indicating the amount of money left after buying the two cheapest chocolates from the sorted list.

Code



```java
import java.util.Arrays;

public class Solution {
    // Calculate the leftover money after buying the two cheapest chocolates
    public int maxLeftover(int[] prices, int money) {
        Arrays.sort(prices); // Sort the prices array
        // Ensure there are at least two chocolates to buy
        if (prices.length < 2 || prices[0] + prices[1] > money) {
            return money; // Return the original amount if not enough to buy two
        }
        return money - (prices[0] + prices[1]); // Subtract the cost of two cheapest chc
    }

    public static void main(String[] args) {
        Solution solution = new Solution();

        // Example 1
        int[] prices1 = {2, 5, 3, 9};
        int money1 = 10;
        System.out.println("Leftover money for Example 1: " + solution.maxLeftover(price
```

Complexity Analysis

- **Time Complexity**: $O(nlogn)$ due to the sorting step, where `n` is the number of chocolates (or elements in the array). This is the dominant factor in the time complexity.
- **Space Complexity**: $O(1)$ for the additional space used (ignoring the space for the input array). This accounts for the constants and minimal extra variables used in the algorithm. If considering the sorting algorithm's space usage, it could be $O(logn)$ for in-place sorting algorithms like QuickSort due to recursive stack space.

← Previous
Buy Two Chocolates (Easy)

Next →
Root Equals Sum Of Children (Easy)

☐ Mark as Completed